Hands-on Exam #2: Poker (contributed by: KMD Kalaw)

1 Problem Specification

1.1 Introduction

Demonstrate what you have learned about structures on top of previous programming concepts learned in class.

Texas Hold 'Em is a popular variant of **poker** wherein each player is dealt two private cards (known as **hole cards**) that belong to them alone, and then during the course of the game, five **community cards** are dealt face-up. A player can then use these shared community cards in conjunction with their own hole cards to make their **best possible poker hand combination**.

Knowledge about playing and the rules of a whole Texas Hold 'Em poker game is not needed. **Your task revolves around identifying certain poker hand combinations**. Identifying poker hand combinations provides a great foundation for manipulating and applying algorithms to data structures. You are given the following assumptions and information about the state of the program:

- Assume that all the provided input is correct and valid (i.e., no need for error handling).
- Input handling and output formatting are already done for you (i.e., no **`scanf()`** nor **`printf()`** in final solution).
- There are 4 players in a game, and the state of the game when you run the unmodified skeleton code is that each player already has 2 hole cards, and the 5 shared community cards are dealt.
- There are about 10 poker hand combinations. You will only be identifying 4 of these. Thus, the provided input will only be from this subset of combinations. Refer to Section 1.4 Poker Hand Combinations for more information.

1.2 Tasks

Your task is to edit the accompanying skeleton file **`POKER-LASTNAME.c`** and define the body of the 5 functions listed below:

- `void combineCards(Card playerCombinedCards[], Card playerHoleCards[], Card commCards[]);`
- `void populateCardMatrix(Card cardMatrix[][N_SUITS], Card combinedCards[]);`
- `char hasFlushCombination(Card combinedCards[]);`
- `int hasOfAKindCombination(Card cardMatrix[][N_SUITS], int N);`
- `void identifyCombination(Player *ptrP);`

1.3 Structures

The following typedefs and structures are provided for you:

```
typedef char String40[41];

typedef struct CardTag
{
  int rank;
  char suit;
} Card;
```

```
typedef struct PlayerTag
{
  String40 name;
  Card holeCards[2];
  Card combinedCards[7];
  Card cardMatrix[13][4];
  String40 bestCombiStr;
} Player;
```

A **Card** contains:

- A rank (**`int`**)
  - The possible values are whole numbers from **1** to **13**, where **1** represents the Ace and **11**-**13** represents the Jack, Queen, and King, respectively.
- A suit (**`char`**)
  - The possible values are the characters **`'c'`**, **`'d'`**, **`'h'`**, and **`'s'`**, which represent clubs, diamonds, hearts, and spades, respectively.
  - There is no ranking among the suits (i.e., there is no ruling about diamonds being stronger than hearts).

A **Player** has:

- A name (**`String40`**)
- Two (2) hole cards (**`Card [2]`**)
- Combined cards (**`Card [7]`**)
  - This is an intermediary structure to help identify the poker hand combination a player has given their personal hole cards and the shared community cards.
  - It is a 1D array of cards comprising 7 cards: the player's personal hole cards (2 cards) and the shared community cards (5 cards).
- A card matrix (**`Card [13][4]`**)
  - This is an intermediary structure to help identify the poker hand combination a player has given their personal hole cards and the shared community cards, sorted from lowest card rank value to highest.
  - It is a 13-row by 4-column 2D array wherein:
    - Each row represents all card rank values in ascending order (lowest to highest; i.e., **1** to **13**).
      - Row index **`0`** represents the Ace card (card rank **1**), row index **1** represents the 2 card (card rank **2**), and so on, until row index **12** which represents the King card (card rank **13**).
    - The columns represents the 4 suits (i.e., there are only 4 Ace cards)
  - Upon initialization, all cards in the card matrix are "null cards" (rank **`0`**, suit **`'x'`**).
- Best combination string (**`String40`**)
  - The string representation of the identified best poker hand combination.

1.4 Poker Hand Combinations

In a game of Texas Hold 'Em, the poker hand combination is determined from a pool of 7 cards (the player's 2 personal hole cards and the 5 shared community cards). The table below shows the 4 combinations you will identify.

Combination	Description	Example
Four Of A Kind	All four cards of the same rank	A♦ A♥ A♠ A♣
Flush	At least five cards of the same suit, but not in a sequence	2♦ Q♦ 10♦ 3♦ 5♦
Three Of A Kind	Three cards of the same rank	10♣ 10♠ 10♥
One Pair	Two cards of the same rank	4♦ 4♠

Again, **note that the inputs that your program will be tested against will only reflect these combinations**. There will be no instance of the output where there are:

- ✘ Two different pairs, known as Two Pair (e.g., 4♦ 4♠ and J♥ J♠)
- ✘ A Three Of A Kind and a One Pair, known as Full House (e.g., 10♣ 10♠ 10♥ and K♠ K♥)
- ✘ No combination (i.e., all outputs will be one of the four combinations in the table above)

Without going into too much detail, the table is ordered from strongest (i.e., best) combination to weakest. This means that if multiple combinations are identified, go for the strongest combination. A more detailed example is presented in Section 5 Example Program Flow.

1.5 Implementation Requirements

Your solution must compile properly with the command:

```
prompt~> gcc -Wall -std=c99 main.c
```

You are **NOT ALLOWED** to do the following:

- ✘ Declare and use global variables (i.e., variables declared outside any function)
- ✘ Use **`exit`**, **`goto`** (i.e., to jump from code segments to code segments), **`break`** (except in **`switch`** statements), or **`continue`** statements
- ✘ Call the **`main()`** function to repeat the process instead of using loops
- ✘ Include libraries or headers other than those already included in the template

Regarding built-in functions in **`string.h`**, you may only use: **`strlen`**, **`strcpy`**, **`strcmp`**, and **`strcat`**

2 Files Provided

- **`POKER-LASTNAME.c`** – The skeleton file containing some initial code you need to complete
- **`main.c`** – Contains the **`main()`** function for testing purposes
- **`defs.h`** – Contains the constants, typedefs, structure templates, and function prototypes for the functions you need to implement
- **`helper.h`** – Contains function prototypes and documentation for convenience/helper functions that you can use in implementing/testing/debugging your solution
  - **`helper.c`** – Contains the implementation of **`helper.h`** (you do not need to open this file)
- **`INPUT#.txt`**– Input file for initializing the structures within the program
- **`EXPECTED#.txt`** – The expected output when the program is run against the corresponding **`INPUT#.txt`**
  - **`INPUT1.txt`** is matched to **`EXPECTED1.txt`**; **`INPUT2.txt`** is matched to **`EXPECTED2.txt`**; and so on

3 Deliverables

Submit/upload the following files before the Canvas submission deadline:

1. **`POKER-LASTNAME.c`** – this file contains your solution
2. **`ACTUAL-LASTNAME.txt`** – this is the actual output redirected text file produced by your own exe file. You can produce this file by running your executable file with input and output redirection in the command line interface as follows:

```
prompt~> a < INPUT1.txt > ACTUAL-LASTNAME.txt
```

4 Testing & Scoring

- Your program will be tested using a different set of input data. Thus, your solution should not be based only on the input data given.
- **Each correct function implementation will be given 10 points each**. Thus, the maximum score will be 50 points.
- Although some functions may depend on prior functions, the checking will be done in isolation.
- **A function that does not produce any of the required output (due to syntax, compilation, or runtime error) will result in a score of 0.**
- **A function should be defined using ONLY the ideas/concepts/syntax/library functions covered so far in CCPROG1 and in Chapters 1-3 of our CCPROG2 Course Notes.** Non-compliance will result in a score of 0 in the worst-case scenario for the offending function definition.

5 Example Program Flow (`INPUT1.txt`)

Step 0. After reading `INPUT1.txt`, the figure above is the initial state of the program.

- The shared community cards are J♥ 10♥ 7♥ J♣ 2♥
- Each player has 2 hole cards
  - Queenie_Hart has: Q♥ 3♥
  - Ace_Ventura has: A♥ K♠
  - Bluff_Daniels has: A♣ 6♠
  - and Jack_Trader has: J♦ J♠

Step 1. For each player, combine their personal hole cards and shared community cards. The visualization of the **combined cards** for each player looks like this:

Player	Combined Cards (Hole Cards + Community Cards)
Queenie_Hart	Q♥ 3♥ J♥ 10♥ 7♥ J♣ 2♥
Ace_Ventura	A♥ K♠ J♥ 10♥ 7♥ J♣ 2♥
Bluff_Daniels	A♣ 6♠ J♥ 10♥ 7♥ J♣ 2♥
Jack_Trader	J♦ J♠ J♥ 10♥ 7♥ J♣ 2♥

Step 2. For each player, map out their combined cards in a **card matrix**.

- Recall that each row represents all card rank values in ascending order (lowest to highest; i.e., **1** to **13**).
- Take for example Jack_Trader's combined cards: Jack_Trader has 1 card with a rank of 2 (2♥), 1 card with a rank of 7 (7♥), 1 card with a rank of 10 (10♥), and 4 cards with a rank of 11 (J♦ J♠ J♥ J♣).
- The visualization for all players looks like this (0x means "null card"):

idx	Queenie_Hart			
0	0x	0x	0x	0x
1	2♥	0x	0x	0x
2	3♥	0x	0x	0x

idx	Ace_Ventura			
0	A♥	0x	0x	0x
1	2♥	0x	0x	0x
2	0x	0x	0x	0x

idx	Bluff_Daniels			
0	A♣	0x	0x	0x
1	2♥	0x	0x	0x
2	0x	0x	0x	0x

idx	Jack_Trader			
0	0x	0x	0x	0x
1	2♥	0x	0x	0x
2	0x	0x	0x	0x

3	0x	0x	0x	0x
4	0x	0x	0x	0x
5	0x	0x	0x	0x
6	7♥	0x	0x	0x
7	0x	0x	0x	0x
8	0x	0x	0x	0x
9	10♥	0x	0x	0x
10	J♥	J♣	0x	0x
11	Q♥	0x	0x	0x
12	0x	0x	0x	0x

3	0x	0x	0x	0x
4	0x	0x	0x	0x
5	0x	0x	0x	0x
6	7♥	0x	0x	0x
7	0x	0x	0x	0x
8	0x	0x	0x	0x
9	10♥	0x	0x	0x
10	J♥	J♣	0x	0x
11	0x	0x	0x	0x
12	K♠	0x	0x	0x

3	0x	0x	0x	0x
4	0x	0x	0x	0x
5	6♠	0x	0x	0x
6	7♥	0x	0x	0x
7	0x	0x	0x	0x
8	0x	0x	0x	0x
9	10♥	0x	0x	0x
10	J♥	J♣	0x	0x
11	0x	0x	0x	0x
12	0x	0x	0x	0x

3	0x	0x	0x	0x
4	0x	0x	0x	0x
5	0x	0x	0x	0x
6	7♥	0x	0x	0x
7	0x	0x	0x	0x
8	0x	0x	0x	0x
9	10♥	0x	0x	0x
10	J♦	J♠	J♥	J♣
11	0x	0x	0x	0x
12	0x	0x	0x	0x

Step 3. Using the player's combined cards and the card matrix, check if the poker hand combinations are present.

- From the combined cards (output of Step 1), what pattern could indicate a **Flush** combination?
- From the card matrix (see Step 2), what pattern could indicate a **Four Of A Kind**, **Three Of A Kind**, or a **One Pair**?

Step 4. Given those checkers, notice that some players have multiple combinations (Queenie_Hart and Ace_Ventura). According to the table in Section 1.4 Poker Hand Combinations, Flush is stronger than One Pair.

Player	**Has Flush Combination?**	**Has N-Of-A-Kind Combination?**			**Final Combination**
	Flush	**Four Of A Kind**	**Three Of A Kind**	**One Pair**	
Queenie_Hart	Q♥ 3♥ J♥ 10♥ 7♥ 2♥			J♥ J♣	Flush
Ace_Ventura	A♥ J♥ 10♥ 7♥ 2♥			J♥ J♣	Flush
Bluff_Daniels				J♥ J♣	One Pair
Jack_Trader		J♦ J♠ J♥ J♣			Four Of A Kind

Step 5. Finally, construct the best combination string.

- Queenie_Hart: **`Flush - h`**
- Ace_Ventura: **`Flush - h`**
- Bluff_Daniels: **`One Pair - J`**
- Jack_Trader: **`Four Of A Kind - J`**